VAS VÁRMEGYEI SZAKKÉPZÉSI CENTRUM
HORVÁTH BOLDIZSÁR
KÖZGAZDASÁGI ÉS INFORMATIKAI
TECHNIKUM

A 5 0613 12 03 számú Szoftverfejlesztő és –tesztelő vizsgaremek

A szoftveralkalmazás dokumentációja

Rendelésfelvétel optimalizálása

Készítették:

GUBACSI TAMÁS

JELENCSICS BÁLINT

KACSÓ MARCELL

SZOMBATHELY

2026

1. Bevezetés

1.1 A program célja

A program célja egy folyamat optimalizálása azáltal, hogy csökkentsük a felesleges mozdulatokat, adatrögzítéseket, illetve pontosítjuk és gyorsítjuk az eljárást és megkönnyítsük a dolgozók munkáját.

A kiválasztott folyamat egy étterem, pontosabban egy pizzéria kiszolgálási területe kezdve a rendelés felvétellel. A folyamat végén az elégedett vásárló pontos és gyors kiszolgálása. Az étterem költségeinek a csökkentése hosszú távon, illetve a környezeti hatás (carbon lábnyom) csökkentése.

Nem elhanyagolható javítási lehetőség továbbá a kommunikáció javítása azáltal, hogy gépi, pontos adatok jutnak el a konyhába.

1.2 Felhasználási terület - célközönség

A programot bármilyen étteremben lehet használni, ahol elektronikus alapú kommunikáció van a pult és a konyha között. Például papír alapú (”bekiabálásos”) üzengetés.

Ez a program csak az étterem dolgozóinak készül, vásárló / megrendelő számára nem elérhető.

1.3 Alap folyamat – kiinduló állapot

Hogy megértsük, hogy miért van szükség az optimalizáláshoz először meg kell értenünk a kiinduló állapotot, hogy hogyan működik most a rendelésfelvétel egy egyszerű, de magas rendelés állománnyal rendelkező pizzériában.

- Rendelés felvétel telefonon / személyesen a pultnál: rendelés rögzítése kézzel írva egy papírcetlire történik a szükséges adatokkal (név, telefonszám, pizza(k), kiszállítási cím).
- Az ’’adathordozó’’ (cetli) továbbítása a konyha felé, ahol értelmezik a rajta lévő adatokat (ha lehetséges).
- Miután a rendelés elkészült a konyhán a cetlit rárögzítik a dobozra (ami a rendelést tartalmazza) és visszaküldik a pulthoz.
- A futár a rendelést a cetli alapján kiszállítja a megrendelőnek

Ez a folyamat rengeteg hibázási lehetőséggel van teli. A legnagyobb lehetséges hiba az adathordozó dekódolása (kézírás olvasása). Ha a konyhán félre olvassák a rendelést nem megfelelő pizza készülhet el. Ha a futár félre olvassa a címet rossz helyre szállítja a pizzát vagy nem tudja elolvasni a telefonszámot, nem éri el a vevőt.

Ezek a hibázások mind megnövekedett kiszállítási időt eredményeznek, ezáltal kevesebb kiszállítás történik és a megrendelők is elégedetlenek lehetnek.
Továbbá a környezeti hatások sem elhanyagolhatóak: átlag napi 800 rendelés = minimum napi 800 cetli. napi 800 cetli = 24 000 cetli havonta = 288 000 cetli évente. Egy 800 darabos post it tömb körülbelül 1500 forint. 360 tömbre van szükség évente, ami 540 000 forint költséget jelent. Csak papír. De tovább lehetne számolni még a toll használatot is.
Nézzük meg még a környezeti hatást! AI szerint egy átlagos fából körülbelül 62 darab post it tömb készül. Tehát az étterem évente átlagosan 5,8 fát pusztít el csak a post it cetlik miatt. Ezek a ''tökéletes'' számok, ahol nincs elírás, felesleges papír pazarlás. Hasonló minta alapján a toll használatot is ki lehetne számolni.

1.4 Hibázási lehetőségek

Legnagyobb hibázási lehetőség a cetlin lévő tartalom félre olvasása és értelmezése. Később problémát okozhat még a visszakövethetőség hiánya esetleges téves rendelésnél. Elveszett cetli pótlása nehézkes, mert semmilyen adatrögzítési rendszer nem áll rendelkezésre.

1.5 Optimalizálás – a program funkciói

- Stabil, gyors kommunikáció a pult és a konyha között: a rendelés rögzítése után a konyhában lévő terminálon megjelennek a szükséges adatok (pizza típusa, mennyisége, plusz feltétek, rendelés beérkezése)
- Konyhai sütések tervezése (FIFO elv): a konyhai terminálon láthatóak a nyitott rendelések, leszűrhetőek a folyamatban lévő sütések, illetve a kész átadásra váró / átadott rendelések nyomon követése
- Futár számára a rendelési adatok kinyomtatása, ezáltal a kézírás olvasási probléma kizárása
- A rendelés felvételnél a korábbi rendelések alapján automatikus kitöltés a rögzítés felületen, ezáltal csökkenthető a rendelés rögzítésének az ideje, továbbá a tévesztés lehetősége is csökken.
- Visszakövethetőség a rendelések oldalon: korábbi átadott rendelések szűrése például reklamációk kivizsgálásához

2. A program

2.1 Felhasználói igények

Átlátható, gyors felület, több eszközön is működjen. Egyszerű kezelhetőség, akár egy új dolgozónak is könnyen kezelhető legyen. Minimalista elképzelés: csak a szükséges információk jelenjenek meg.

2.2 Tervezés: projekt tagok

- **Gubacsi Tamás**: 15 éve különböző gyárakban dolgoztam itt Szombathelyen. Általában Minőségbiztosítási vonalon problémák megoldásán és folyamatok optimalizálásán dolgoztam. Jelenleg Gyártásellenőrzési csoportvezető vagyok. Programozást terén először a gyártástámogató segédszoftverek terén szereztem tapasztalatot. Későbbiekben hobbiból Home Assistant okosotthon rendszert építettem ki magamnak, különböző energiahatékony optimalizálásokkal.
- **Jelencsics Bálint**: Több mint 10 éve dolgozom szerelőként egy adott gyárban itt Szombathelyen, így jelentős gyakorlati és műszaki tapasztalattal rendelkezem. A Ruandersnél elvégzett Programozás alapjai tanfolyam segítségével fektettem le a szoftverfejlesztői ismereteim alapjait. Szabadidőmben autódiagnosztikai szoftverek telepítésével és kiértékelésével foglalkozom.
- **Kacsó Marcell**: 4 éve gyárakban dolgozom, Szombathelyen és a közeli településeken, mint Sárvár, Szentgotthárdon, kipróbáltam magamat a konyhában is, a McDonald'sban, valamint kipróbáltam a pizzafutár állást is a Magic pizzériánál. Jelenleg Szentgotthárdon operátorként dolgozom, a Pannonworkben, a G&P Quality Managementnek egyszerűsített foglalkoztatásban dolgozva. A munkát illetően a terveim a videójátékfejlesztés felé tendálnak az Unity játékmotort használva.

2.3 Tervezés: feladatok felosztása:

- Gubacsi Tamás: adatbázis modell tervezése, létrehozása. Dokumentáció elkészítése, tesztelések végrehajtása. Végső optimalizáció a programon (Frontend – Backend – Adatbázis kommunikáció). Többiek feladatainak az ellenőrzése és koordinálása.
- Jelencsics Bálint: Frontend dizájn tervezése, felépítése. Felület létrehozása React keretrendszerben. Többiek feladatainak az ellenőrzése. Kommunikáció fenntartása a többiekkel.
- Kacsó Marcell: Backend megtervezése majd létrehozása C#-ban. Többiek feladatainak az ellenőrzése.

Feladat / Név	Gubacsi Tamás	Jelencsics Bálint	Kacsó Marcell
Adatbázis tervezése, létrehozása	X		E
Frontend tervezése, létrehozása	E	X	E
Backend tervezése, létrehozása	E	E	X
Dokumentáció elké-szítése	X	E	E
Tesztelések	X	X	E
Adatbázis - Backend - Frontend szinkronizá-lása	X		

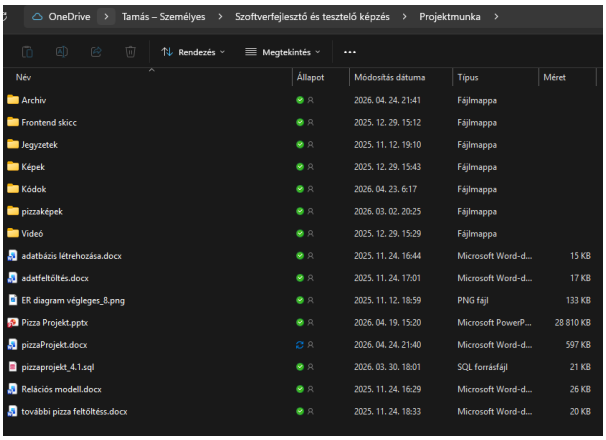
X - elvégezte

E- ellenőrizte

1. táblázat - feladatok

2.4 Kapcsolattartás módszere

A legfőbb kapcsolattartási módszer a személyes találkozó volt az iskolában. Ha ez nem volt lehetséges, akkor a telefonos kommunikáció jött még elő. Később Facebook Messengeren szöveges, illetve telefonos konferencia. Képernyőmegosztásokhoz Discord-ot használtunk, ahol nem csak a képernyőt tudtuk megosztani, hanem kisebb fájlokat is tudtunk küldeni egymásnak. A fájlok tárolásához és megosztásához OneDrive-ot használtunk. Itt hoztunk létre a Github-on kívül egy további biztonsági mentést és verzió követést archiválások által.



1. ábra - OneDrive felhő

3. Adatbázis – ER modell megtervezése

Először összeszedtük, hogy milyen adatokat rögzítenek rendelésenként az étteremben (telefonos rendelés lett alapul véve):

- Kiszállítási cím (utca, házszám), rendeléseket csak Szombathelyre vesznek fel
- Vásárló adatai: név és telefonszám
- Rendelés adatai: milyen és mennyi pizza, fizetési mód

Ezek az adatok alapján elkezdtük szétbontani, hogy milyen adatokat kell / lehet rögzíteni az adatbázisban:

- Kiszállítási cím: irányítószám, város, utca, házszám, emelet, ajtó
- Vásárló adatai: név, telefonszám
- Rendelés / vásárlás adatai: dátum, milyen pizza, mennyiség, fizetési mód

Kell egy adatbázis a lehetséges pizzákról:

- Alap pizzák: név, ár
- Feltétek: milyen feltétek vannak a pizzán (, hogy később ezeket ki lehessen egészíteni), és ezek árai

Az adatok alapján minimum 5 táblára szükség van:

1. Kiszállítási hely
2. Vásárló
3. Rendelés
4. Pizza
5. Feltétek

Minden egyes táblába kell egy főkulcs:

- Cím Id
- Rendelő azonosító
- Rendelési Id
- Pizza Id
- Feltét Id

A fő táblákat össze kell kapcsolni, emiatt létre kell hozni 4 kapcsolótáblát:

1. Pizza feltétek kapcsolótábla: Feltétek és Pizza táblát kapcsolja össze
2. Tartalmaz kapcsolótábla: Pizza és Rendelés táblát kapcsolja össze
3. Lead kapcsolótábla: Rendelés és Vásárló táblát kapcsolja össze
4. Kéri kapcsolótábla: Vásárló és Kiszállítási hely táblát kapcsolja össze

Kapcsolási módok az adatbázisban:

1. Feltétek és Pizza közötti kapcsolat: több-több (N:M) kapcsolat. Egy feltét több pizzán is szerepelhet és egy pizzán több fajta feltét is lehet.
2. Pizza és rendelés közötti kapcsolat: több-több (N:M) kapcsolat. Egy rendeléshez több pizzát is tartalmazhat, illetve egy pizza több rendeléshez is kerülhet.
3. Rendelés és Vásárló közötti kapcsolat: egy-több (1:N) kapcsolat. Egy vásárlóhoz több rendelés is tartozhat, de egy rendeléshez csak egy vásárló lehet.

4. Vásárló és Kiszállítás hely közötti kapcsolat: több-több (N:M) kapcsolat. Egy vásárlóhoz több cím is tartozhat és egy címen több vásárló is lakhat.

A fenti adatok alapján a Relációs modellt létre is lehetett hozni:

Kiszállítási hely (Cím_Id, Város, Irányítószám, Utca, Házszám, Emelet/ajtó)

Kéri (Cím_Id, Rendelő azonosító)

Vásárló (Rendelő azonosító, Név, Telefonszám)

Lead (Rendelő azonosító, Rendelési_Id)

Rendelés (Rendelési_Id, Mennyiség, Dátum idő, Fizetési mód, Státusz, Rendelő azonosító)

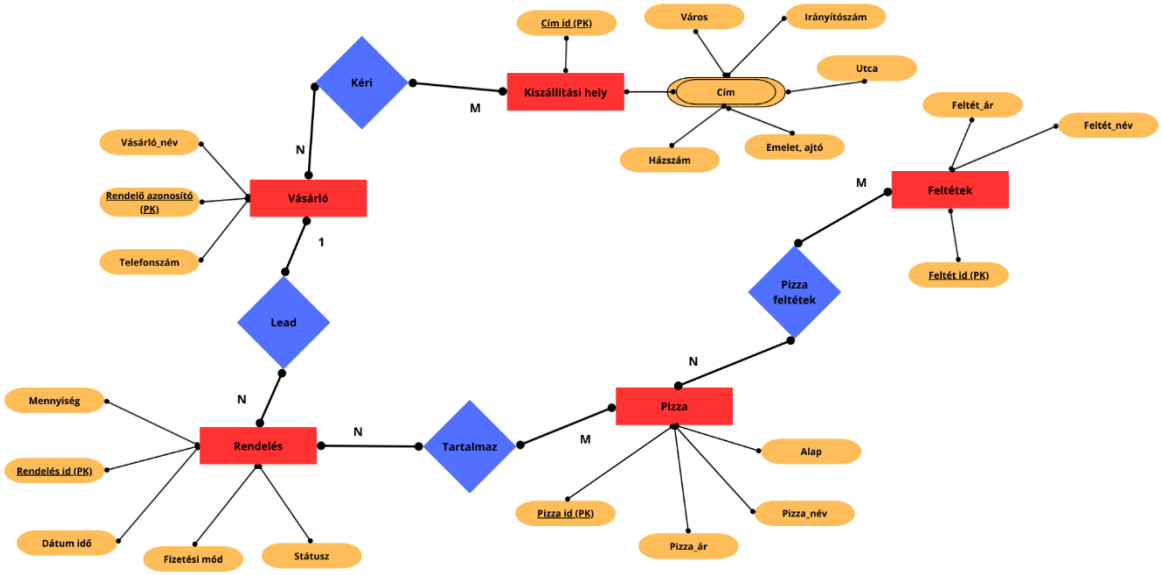
Tartalmaz (Rendelési_Id, Pizza_Id)

Pizza (Pizza_Id, Pizza_ár, Név, Alap)

Pizza feltétek (Pizza_Id, Feltét_Id)

Feltétek (Feltét_Id, Feltét_ár, Név)

3.1 ER modell:



2. ábra – ER Modell

3.2 Az adatbázis cellái

Kiszállítási hely:

Cím_Id = kiszállítási cím azonosítója

Város = kiszállítási cím városa

Irányítószám = kiszállítási cím irányítószáma

Utca = kiszállítási cím utcája

Házszám = kiszállítási ház száma

Emelet/ajtó = kiszállítási cím emelete / ajtaja

Vásárló:

Rendelő azonosító = megrendelőnek az azonosítója

Rendelő_név = megrendelőnek a neve

Telefonszám = megrendelőnek a telefonszáma

Rendelés:

Rendelési_Id= rendelésnek az azonosítója

Mennyiség = rendelési mennyiség

Dátum idő = rendelési dátum és idő

Fizetési mód = a rendelésnek a fizetési módja (Bankkártyás, SZÉP kártyás, készpénz)

Státusz = rendelés állapota (Rendelés rögzítve, Rendelés készül, Rendelés kiszállítás alatt, Rendelés lezárva)

Pizza:

Pizza_Id = pizza azonosítója

Pizza_ár = pizzának az ára

Pizza_név= Pizzának a neve

Alap = pizzának az alap szószja

Feltétek:

Feltétek_Id = feltéteknek az azonosítója

Feltét_név = feltéteknek a neve

Feltét_ár = feltéteknek az ára

3.3 Adatbázis létrehozása SQL-ben

Create database Pizzaprojekt;

CREATE TABLE Feltetek (Feltet_Id INT PRIMARY KEY, Feltet_ár DECIMAL(10, 2) NOT NULL, Név VARCHAR(100) NOT NULL);

CREATE TABLE Pizza (Pizza_Id INT PRIMARY KEY, Pizza_ár DECIMAL(10, 2) NOT NULL, Név VARCHAR(100) NOT NULL, Alap VARCHAR(100));

CREATE TABLE Pizza_feltetek (Pizza_Id INT NOT NULL, Feltet_Id INT NOT NULL, PRIMARY KEY (Pizza_Id, Feltet_Id), FOREIGN KEY (Pizza_Id) REFERENCES Pizza(Pizza_Id), FOREIGN KEY (Feltet_Id) REFERENCES Feltetek(Feltet_Id));

CREATE TABLE Kiszállítási_hely (Cím_Id INT PRIMARY KEY, Város VARCHAR(100) NOT NULL, Irányítószám VARCHAR(10) NOT NULL, Utca

VARCHAR(100) NOT NULL, Házszám VARCHAR(20) NOT NULL, Emelet_ajtó VARCHAR(50));

CREATE TABLE Vásárló (Rendelő_azonosító INT PRIMARY KEY, Név VARCHAR(150) NOT NULL, Telefonszám VARCHAR(20));

CREATE TABLE Rendelés (Rendelési_Id INT PRIMARY KEY, Mennyiség INT NOT NULL, Dátum_idő TIMESTAMP NOT NULL, Fizetési_mód VARCHAR(50) NOT NULL, Státusz VARCHAR(50) NOT NULL, Rendelő_azonosító INT NOT NULL, FOREIGN KEY (Rendelő_azonosító) REFERENCES Vásárló(Rendelő_azonosító));

CREATE TABLE Lead (Rendelő_azonosító INT PRIMARY KEY, Rendelési_Id INT NOT NULL, FOREIGN KEY (Rendelő_azonosító) REFERENCES Vásárló(Rendelő_azonosító), FOREIGN KEY (Rendelési_Id) REFERENCES Rendelés(Rendelési_Id));

CREATE TABLE Tartalmaz (Rendelési_Id INT NOT NULL, Pizza_Id INT NOT NULL, PRIMARY KEY (Rendelési_Id, Pizza_Id), FOREIGN KEY (Rendelési_Id) REFERENCES Rendelés(Rendelési_Id), FOREIGN KEY (Pizza_Id) REFERENCES Pizza(Pizza_Id));

CREATE TABLE Kéri (Cím_Id INT NOT NULL, Rendelő_azonosító INT NOT NULL, PRIMARY KEY (Cím_Id, Rendelő_azonosító), FOREIGN KEY (Cím_Id) REFERENCES Kiszállítási_hely(Cím_Id), FOREIGN KEY (Rendelő_azonosító) REFERENCES Vásárló(Rendelő_azonosító));

3.4 Adatbázis feltöltése

Az adatbázis feltöltéséhez az étterem honlapját vettük alapul, ahol az összes pizza fajtát, feltételeket, alapokat összegyűjtöttük és egyesével fel lettek töltve SQL-en keresztül.

Feltöltési példák:

INSERT INTO Pizza (Pizza_Id, Pizza_ár, Név, Alap) VALUES

(1, 2350.00, 'Mint a villám pizza', 'paradicsomos alap'),

(10, 2550.00, 'Nagymenők pizza', 'paradicsomos alap'),

(12, 2650.00, 'Mad Max pizza', 'paradicsomos alap'),

(13, 2650.00, 'Blöff pizza', 'paradicsomos alap'),

(16, 2650.00, 'Ananász expressz 1. pizza', 'paradicsomos alap'),

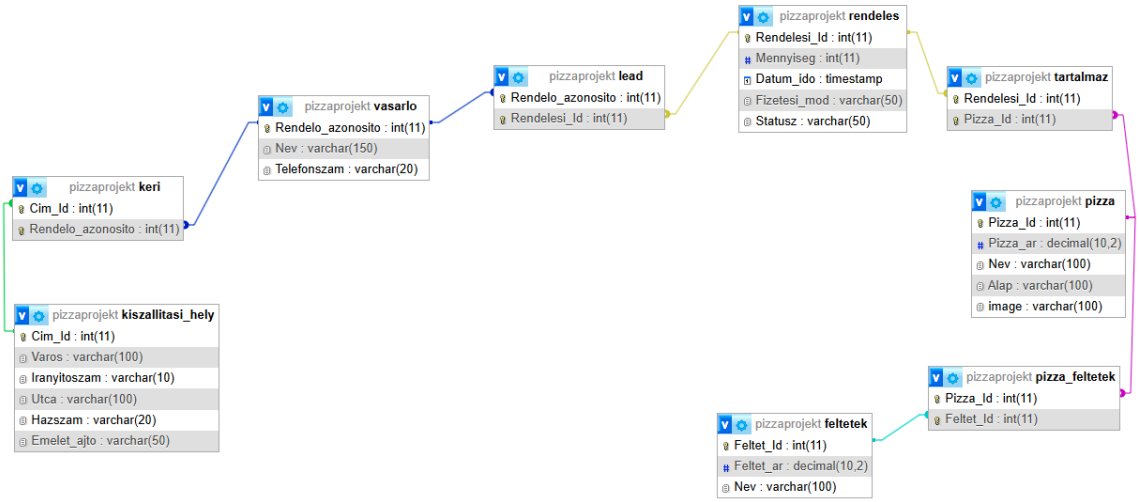
(17, 2650.00, 'Ananász expressz 2. pizza', 'paradicsomos alap');

INSERT INTO Pizza_feltetek (Pizza_Id, Feltet_Id) VALUES (1, 1), (10, 8), (10, 2), (10, 1), (12, 21), (12, 3), (12, 13), (12, 4), (12, 1);

A Backend felépítése miatt az összes többi táblába is töltöttünk adatokat, hogy a projektmunka elején a lekérdezéseket ki tudjuk próbálni. Néhány példa:

INSERT INTO Kiszállítási_hely (Cím_Id, Város, Irányítószám, Utca, Házszám, Emelet_ajtó) VALUES (101, 'Szombathely', '9700', 'Fő tér', '1.', 'Fsz. 1.'), (102, 'Szombathely', '9700', 'Hunyadi János utca', '14.', NULL);

3.5 A kész adatbázis



3. ábra - adatbázis

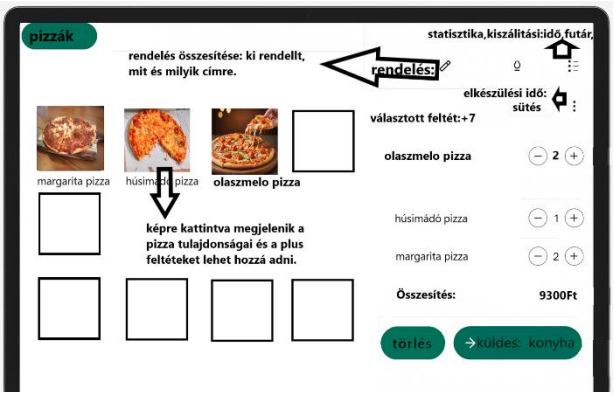
4. Frontend

4.1 Frontend tervezése

A projekt tervezésének a munkálatai párhuzamosan folytak (adatbázis/backend/frontend) az adatbázis tervezés után inkább a Frontenddel folytatnánk.

Először összeszedtük, hogy milyen szempontok szerint szeretnénk, hogy működjen a felhasználói felület:

- 3 fő felület:
    - Rendelő felület: rendelések rögzítéséhez
    - Konyha felület: konyhában nyomon lehessen követni a pizzák elkészültségét
    - Kész rendelések felület: a kiszállításhoz szükséges adatok böngészése
- a honlap működjön különböző képernyőmérettel rendelkező eszközökön:
    - Desktop: asztali számítógép
    - Tablet: közepes méretű kijelzőn
    - Mobil: kis kijelzős kézi eszközön
- egérrel és érintőképernyővel is lehessen kezelni a honlapot



4. ábra - kezdeti terv

- kezdőoldalon az összes pizza szerepeljen: ne kelljen keresni
- a tételek olyan sorrendben jelenjenek meg, mint az étlapon
- plusz feltételeket (kiegészítőket) lehessen a kosárban hozzá adni
- automatikus cím kitöltés: ne kelljen minden egyes rendelésnél beírni a címet, adatbázisból (korábbi rendelések alapján) visszakereshetőek a címek
- rendelésenként egyszerűen látható legyen a kiszállításhoz szükséges adatok
- konyha oldalon átlátható legyenek a pizzák készültségi állapotai:
    - sütésre váró pizzák
    - sütés alatt lévő pizzák
    - elkészült (sütés kész) pizzák
    - átadásra váró pizzák (becsomagolva / dobozolva)

4.2 Frontendhez használt programok, keretrendszerek

A frontend elkészítéséhez Visual Studio Code-ot (VS Code) használtunk REACT keretrendszerrel.

A VS Code az egyik leg elterjedtebb ingyenes kódszerkesztő alkalmazás. Továbbá beépített Git rendszer támogatással rendelkezik, ezáltal könnyebbé téve a programozók munkáját és elősegíti a verziók nyomon követesét.


```
[
  {
    "id": 350,
    "datum": "2026-04-23 19:02",
    "cim": "Személyes átvétel (pultnál),",
    "rendeloNev": "Pult",
    "telefonszam": "Nincs telefonszám",
    "fizetesiMod": "keszpenz",
    "statusz": "átvehető",
    "osszeg": 4250,
    "sutesiIdo": null,
    "tetelek": [
      {
        "pizzaNev": "Legenda vagyok pizza",
        "mennyiseg": 1,
        "fizetes": "keszpenz",
        "tetelOsszeg": 4250,
        "feltetek": [
          {
            "nev": "sajt",
            "ar": 500
          },
          {
            "nev": "kockabacon",
            "ar": 400
          },
          {
            "nev": "sonka",
            "ar": 400
          },
          {
            "nev": "füstölt sajt",
            "ar": 500
          }
        ]
      }
    ]
  },
```

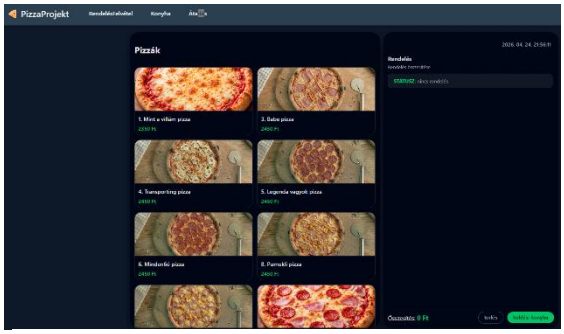

5. ábra - Json adat

A REACT előnye, hogy a felhasználói felületet több újra felhasználható komponensekre bontja. Ez a felépítés segít átláthatóbbá tenni a (HTML szerű) kódot, ami segít a fejlesztési idő csökkentésében. A REACT Virtual DOM-ot használ, ezáltal a kódban való változtatásokat azonnal láthatjuk és módosíthatjuk, ha szükséges.

Az oldalak közti navigációhoz React Router DOM-ot használtunk. Ez a rendszer teszi lehetővé a Single Page Application (SPA) architektúrát, ahol a böngésző a navigáció során nem tölti újra a teljes weboldalt.

A VS Code és a REACT nagyon jó párosítás, mert a HMR (Hot Module Replacement) -nek hála minden egyes mentés után azonnal újra töltődik a honlap és a változásokat egyből megjeleníti.

4.3 Kezdőlap – Rendelés rögzítés



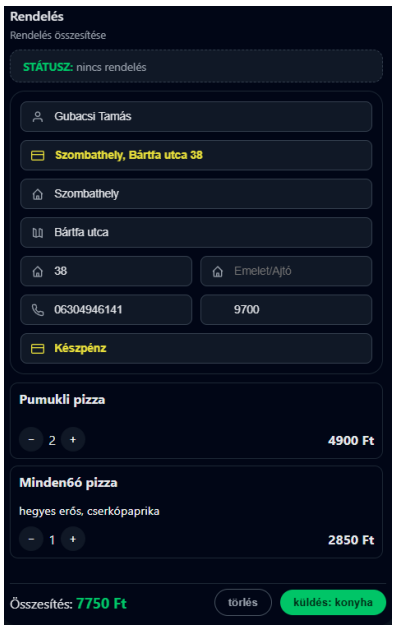
6. ábra - Főoldal

A kezdőlap alappillére a Fetch metódus (JavaScript). Ezzel a módszerrel kérjük le és töltjük be a pizza fajtákat a honlapon. A Fetch metódussal kérjük le és töltjük be az adatokat a szerverről. A legnagyobb előnye, hogy promise alapú (aszinkron) működés, ami segíti a honlap fluid (folyamatos) működését. Ezáltal a honlap betöltése gyorsabb. A fetch kommunikációhoz JSON formátumot használtunk, amit a backend küld a honlapnak. Lekérdezéskor a JSON fájlba lista szerűén jelennek meg a pizzákról az adatok.

7. ábra - Feltét választó

A kezdőlapon a pizza kiválasztása után ki lehet még választani a plusz feltéteket, ha a rendelőnek erre van plusz igénye. Természetesen a folyamat gyorsítása végett lett csinálva egy „Csak a Pizza” gomb is, ezáltal plusz feltét nélkül lehet rögzíteni a rendelést.

A kezdőlapon nem csak a pizza típusok szerepelnek, hanem a rendelésfelvétel is itt történik. Emiatt nem csak GET Request törtink itt, hanem POST Request is. A frontend küldi az adatokat a Backendnek és az továbbítja az adatbázisba. Korábbi specifikáció volt, hogy minden rendelést rögzítenünk kell az adatbázisba. Mivel a fő célunk a rendelés folyamat gyorsítása / egyszerűsítése elengedhetetlen volt a rendelés rögzítésénél a korábbi rendelők visszakereshetősége. A backendben használt vezérlőnek hála a REACT-ben meg lehetett oldani, hogy a rendelőre (név+telefonszám páros) a backend visszakeresse az adatbázisból a rendelő címét. A lekérdezés után már könnyen integrálható volt, hogy a rendelőt és a címeket automatikusan javasolja a felhasználónak. A későbbi tesztelések során előjött az, hogy a rendszer hogyan kezelje, amikor egy rendelőnek több címe is van az adatbázisban. Először az összes lehetséges címet megjelenítette és a felhasználó a név beírása közben tudta kiválasztani a szükséges címet. Ez a megoldás 4 címig jól működött. Későbbi megoldás lett, hogy a javaslat csak a rendelőre vonatkozik (név alapján ke-



res és telefonszám alapján párosít) és csak Név + Telefonszámot kell kiválasztani először. Ilyenkor megjelenik egy plusz sor, ahol ki lehet választani több címet is (ha van a rendelőhöz párosítva). Alapértelmezetten ABC sorrendben az első címet tölti be teljesen. Váltáskor a kiválasztott cím adatai betöltődnek, hogy a felhasználó ellenőrizni tudja a rendelővel.

Miután kitöltöttük a rendelő adatait még lehetséges a rendelés tartalmának (pizzák) a módosítása. További pizzákat, feltételeket, illetve mennyiségeket lehet még beállítani a véglegesítés előtt. De akár az egész rendelés törlése is lehetséges (ez a gomb csak a frontendbe beírt adatokat törli!).

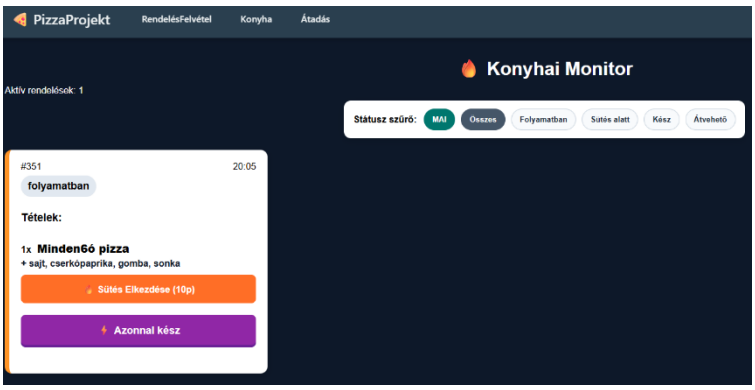
8. ábra - rendelés rögzítés

A rendelés rögzítése a Küldés: Konyha gombbal lehetséges.

Ilyenkor az adatbázisnak továbbítja a rendeléshez szükséges összes adatot, ami végül az adatbázisban köt ki.

4.4 Konyha – készültségi állapotok

A főoldal után – amit a pultnál tudnak használni – jöhet a konyhában használatos felhasználói felület. A konyhai oldal alapja ugyan az, mint a főoldal. Itt is Fetch metódust használtunk.



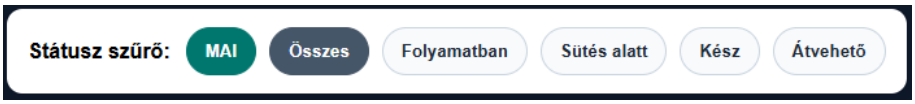
9. ábra - Konyha oldal

Konyhában használható oldal lényege, hogy a séf egyszerűen és gyorsan átláthassa a különböző rendelésekben leadott pizzák készültségi állapotát.

Először meghatároztuk, hogy milyen készültségi állapotokat szeretnénk a konyhai oldalon:

- Folyamatban: az frissen beérkezett rendeléseket mutatja, ahol még nem kezdték el a sütést, de a rendelés már rögzítve lett
- Sütés alatt: amikor látszik, hogy a szakács a pizzát már berakta a kemencébe, és a sütés folyamata elindult
- Kész: sütés elkészült, de még nem lett bedobozolva / tányérra rakva, hogy ki lehessen vinni a pulthoz
- Átvehető: a pizza be van dobozolva / tányérra van készítve, ezáltal a pulthoz kiadható

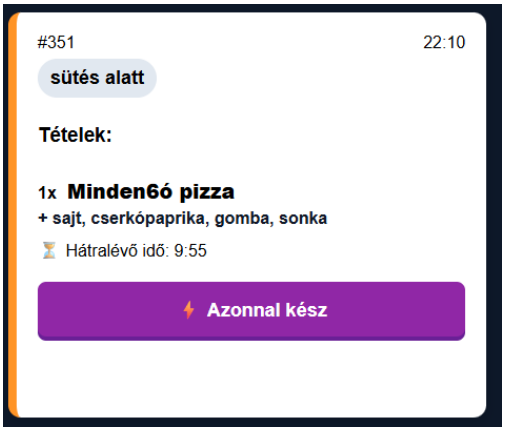
Különböző szűrések tesztelése után hamar rájöttünk,



10. ábra - státusz szűrők

hogy szükségünk lesz egy olyan felületre vagy szűrésre, ahol az összes rendelési állapotot látjuk. Így született meg az „Összes” szűrési feltétel, ahol minden rendelési állapot látható. További plusz igényt kaptunk az étteremben dolgozóktól, hogy jó lenne, ha a különböző szűréseknél akár egyszerre több szűröt is be lehessen állítani. Eleinte lenyíló szűrőt próbáltunk ki, de ennél a plusz kérést nem lehetett megvalósítani. Hamar eljutottunk arra az állapotra, hogy egyszerű gombok legyenek a szűrők, amik mindig egyszerre láthatóak. A kiválasztott szűrők gombjait különböző interaktív színekkel láttuk el, hogy gyorsan és könnyen értelmezhető legyen, hogy mely szűrők vannak bekapcsolva. Ezeket a szűrési opciókat viszonylag egyszerűen bele lehetett integrálni a rendszerbe.

Ezen az oldalon a leendő felhasználótól kaptunk egy másik javaslatot is: a szakácsnak nem lesz arra ideje, hogy pár percenként oda lépjen és mindig nyomkodja, hogy mikor kezdődik és mikor ér véget egy pizza sütése. A probléma megoldásán sokat ötleteltünk. Az étterem elmondása szerint egy átlagos pizza sütési ideje 8 perc. Tehát elméletben 8-9 percenként rakják a sütőbe a pizzákat. 4 kemencéjük van és csúcsidőben átlag 2-en dolgoznak a konyhán. Míg egy pizza bent van 1 sütőben, addig a többi pizza rendelést készítik elő. A lényeg számunkra, hogy a sütés kezdésének a jelzését nem tudjuk kiküszöbölni. De a sütés elindítása után már megoldható, hogy automatikusan készre jelentődön a pizza. Elméletileg, ha a szakács vagy a kisegítő a pizza kemencébe helyezése után a programban elindítja a sütést, akkor elég csak egy számlálót indítani, ami után automatikusan készre jelentődik a sütés. Megfigyelések alapján a gomb megnyomása után 10 perc már kiveszik a pizzát a kemencéből.



11. ábra - Visszaszámláló

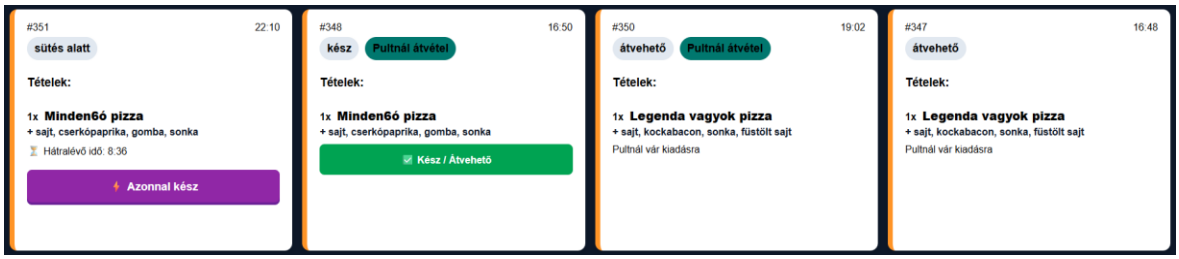
Eleinte a Frontend programban létrehoztunk egy változót, ahol rögzítjük a sütés elindításának az idejét, majd egy egyszerű számláló elindult, ami 10 perc múlva készre jelentette. Sajnos ezzel a megoldással több probléma is felmerült. Mivel a Frontend-ben hoztuk létre a változót és a számláló ez alapján ment, minden egyes oldal újra betöltésé vagy szűrő változtatása után a számláló újra indult az elejétől. Ez a megoldás csak akkor működött, ha a sütés elindítása után nem nyúlunk a honlaphoz. Egyértelműen nem volt jó ötlet ez a megoldás. Tovább gondolva a problémát arra jutottunk, hogy a Backendben kell a sütés kezdetének az időpontját rögzíteni és a Frontendben megcsinálni, pontosabban átalakítani a visszaszámlálót. Természetesen ilyenkor a Frontend kommunikációját ki kellett egészíteni egy plusz adattal, hogy a Backendben

lévő változót elérje. A kísérletezgetések közben felmerült még, hogy miért a sütés kezdetét rögzítjük? Átgondolva a helyzetet a Frontendben egyszerűbb leprogramozni, hogy mennyi idő van még hátra egy megadott időhöz képest. Így arra jutottunk, hogy a Backendben lévő változóba a sütési időnek a végét rögzítjük, ezáltal egyszerűbbé vált a program, a Frontendben lévő függvény először azt figyeli, hogy a rögzített idő elmúlt-e már? Ha elmúlt, akkor a sütés kész, ha nem akkor kiszámolja, hogy még mennyi idő van hátra. Ezzel a logikával meg lehetett oldani, hogy ha akár bezárjuk a honlapot vagy frissítjük mindig pontosan mutatja, hogy mikor készül el a sütés.

A Konyha oldalnak van még egy fontos funkciója: rendet teremteni a rendelések káoszában. A képernyőn megjelenő rendeléseknek fel kellett építeni egy logikát, amit a felhasználók használni és értelmezni tudnak. Hosszú mérlegelések után arra jutottunk, hogy az elsődleges sorrend az alábbiak szerint alakul:

1. Folyamatban lévő rendelések: rendelés rögzítve, sütés még nincs elkezdve
2. Sütés alatt: sütés el lett indítva, de még nem járt le a 10 perc (ez az adat igazából csak tájékoztatás a szakács vagy a kisegítő számára)
3. Kész állapot: kemencéből már kivett pizza, de még túl forró a dobozoláshoz, illetve még nincs vagy nem volt idő a dobozolásra
4. Átvehető státusz: dobozolás / tálalás kész, mehet ki a pulthoz

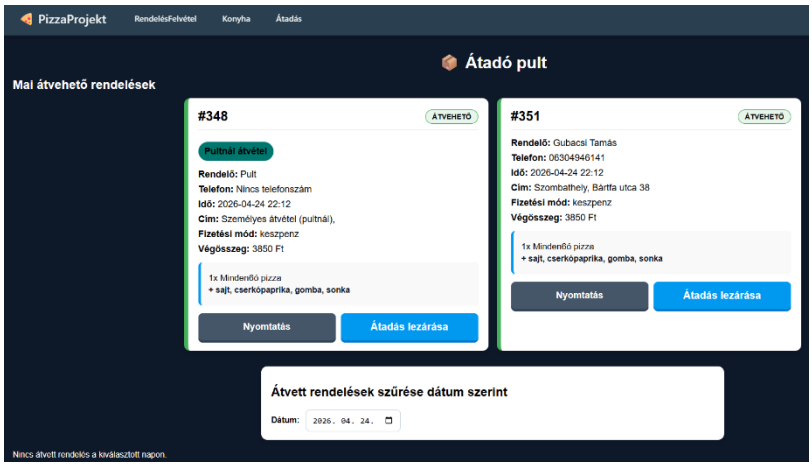
A státuszokon belül pedig a rendelési azonosító sorszáma a sorrend. A rendelési azonosító egy egyszerű számláló: minél régebbi a rendelés annál kisebb a száma. Ez a két logika alapján akár ránézésre is gyorsan megállapítható, hogy most melyik pizzával kell foglalkozni.



12. ábra - feladatok rendezése

4.5 Rendelések átadás - kiszállítás

Elérkeztünk az utolsó felhasználói felülethez, ahol a kész rendeléseket láthatjuk. Ennél az oldalnál a célunk az volt, hogy a pultos azonnal lássa a rendelésenkénti szükséges információkat. A fő célunk itt az, hogy látszódjon egy vagy több rendeléshez az összes szükséges információ:



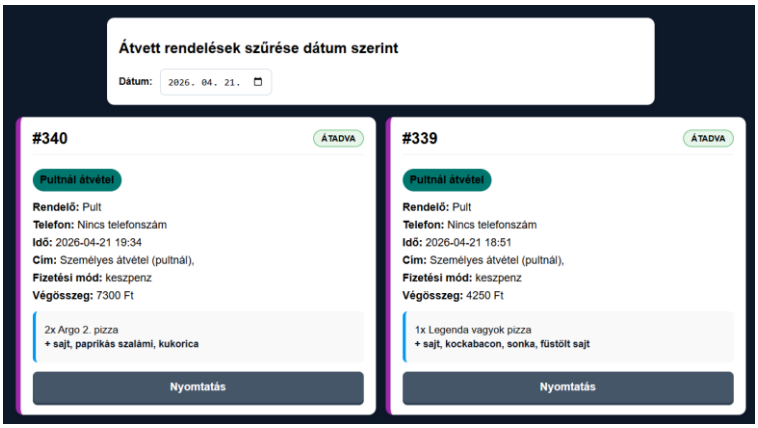
13. ábra - Rendelések

- megrendelő neve: a téves szállítások elkerülése miatt
- megrendelő telefonszáma: hogy a futár el tudja érni a megrendelőt kiszállításkor
- rendelés dátuma: saját adminisztrációnak
- kiszállítási cím: futár számára
- fizetési mód: futár számára, hogy kell-e vinnie kártya olvasót
- végösszeg
- milyen és mennyi pizza (esetleges plusz feltétekkel)

14. ábra - Kiszállítási adatok

A tesztelések során hamar láthatóvá vált, hogy ezen az oldalon idővel egyre nagyobb káosz lett a sok információ miatt. Minél több rendelés került rögzítésre, annál több adat jelent meg. Gyakran a sok információ nagyobb hátrány, mint a kevés információ. Keveredésre ad okot.

Emiatt először létrehoztunk egy dátumszűrőt, amit alapértelmezetten az aktuális napra állítottunk. Ezután beállítottuk, hogy csak azokat a rendeléseket jelenítse meg, amik az aktuális dátummal rendelkeznek (csak az aznapi kiszállítást). Ha már dátum szűrőt raktunk bele, akkor már adta magát, hogy visszakereshetőség is legyen. A dátum selector segítségével bármilyen dátumot ki lehet választani és a selector alá megjelennek a kiválasztott dátumnál rendelt rendelések és azok adatai.



15. ábra - korábbi rendelések

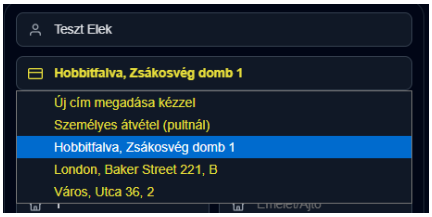
A dátum kiválasztó fölé mindig az átadásra váró kész pizzák rendelései jelennek meg. Ezáltal látszik, hogy mennyi még át nem adott rendelés várakozik. Az átadott rendeléseknek csináltunk egy „átadott” gombot, amivel lezárjuk a rendelést és át kerül a dátumválasztó alá. Plusz igény volt az étteremtől, hogy legyen egy nyomtatás funkció a rendelések számára. Szerencsére a nyomtatás funkciót egyszerűen meg lehet oldani JavaScript segítségével. A felesleges dolgokat, elemeket pedig a css-ben könnyen be lehetett állítani, hogy a nyomtatási képen ne látszódjanak.

16. ábra - Nyomtatási kép

4.6 Végső igazítások

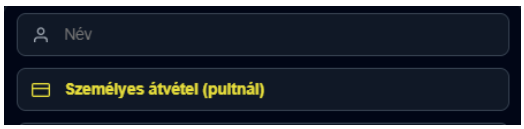
„Ember tervez egér végez”

Természetesen miután azt hittük, hogy mindenre is gondoltunk utána jöttek a honlapon szükséges finom hangolások, hogy a megadott funkciók jobban működjenek. Először a rendelés oldalnál kellett módosítanunk: mi történik akkor, ha a rendelőnek (aki már korábban rendelt



17. ábra - címek

az étteremből) több kiszállítási címe is van. Ezt igazából egy egyszerű legördülő menüvel meg lehetett oldani.

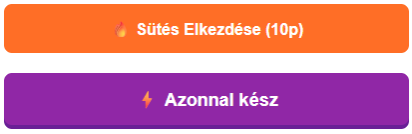


18. ábra - plusz funkciók

Később rájöttünk, hogy szükség lesz egy „személyes átvétel” opcióra. Gyakran van olyan, hogy a megrendelő a hosszú kiszállítási idő miatt személyesen az étteremben szeretné átvenni a rendelését. Ilyenkor nem szükséges kitölteni (tehát lehet üresen hagyni) a kiszállítási cím mezőket. Először ezt egyszerűnek véltük megcsinálni. Eleinte csak arra gondoltunk, ha a megrendelő telefonon rendeli. Jogosan merült fel később az a lehetőség, hogy ha a megrendelő személyesen megy el megrendelni az ételt elvitelre. A legegyszerűbb megoldás az lett, hogy ilyenkor „dummy” adatokat mentünk el automatikusan, ami a felhasználói oldalról nem látszódik. Pl.: Rendelő: Pult, Telefonszáma: nincs. Ezáltal nem kell kikapcsolni az ellenőrzést és az adatbázist / backendet sem kellett utólag módosítanunk.

A Konyha oldalon csináltunk egy gyors és alapértelmezett szűrést, hogy ne az összes eddigi rendelést listázza ki, hanem csak az aktuális napi feladatokat. Továbbá csináltunk egy „Azonnal kész” gombot is, ha a szakács esetleg elfelejtette elindítani a számlálót.

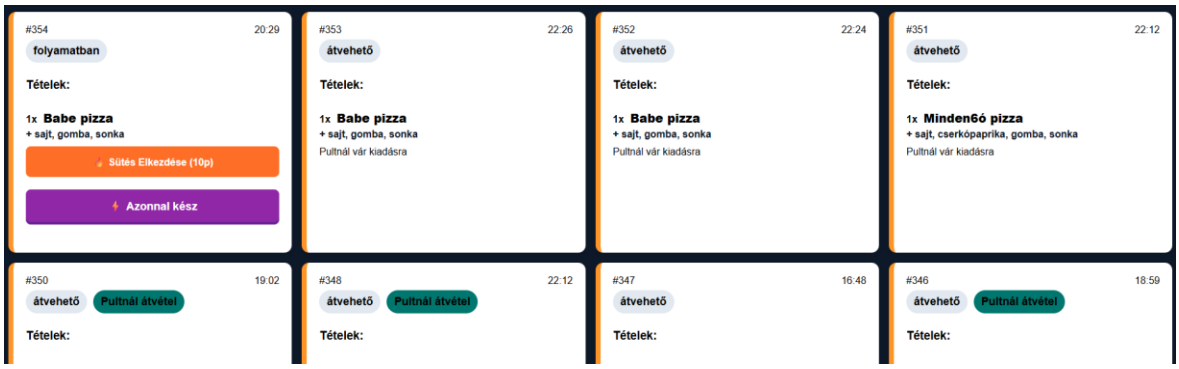
Természetesen először csak a funkciókkal foglalkoztunk, az oldal kinézetével nem. Így utólag elég sok csinosítgatásokra volt szükség. Mindenhol javítottunk az olvashatóságon. Például a Konyha oldalon a felesleges rendelési adatokat, nagyobb betűket alkalmaztunk a pizzák kilistázásához.



19. ábra - szinezések

Az összes gombot nagyítottuk és színeztük, hogy érintő kijelzőkön is jól és könnyen használható legyen.

Az információdömpinget is igyekeztünk csökkenteni azáltal, hogy csökkentettük a képernyől lévő információk mennyiségét. Például a Konyha oldalon egy sorban csak 4 rendelés látható. Továbbá egy rendelés „dobozának” meg lett határozva egy minimum szélesség és magasság, hogy egységes legyen a kinézet. A Rendelések oldalon is csökkentettük az egy sorban megjelenő rendelések számát. Itt csak 2 rendelés jelenik meg egy sorban, mert több információ van egy dobozban. Szintén be lett állítva egy minimum szélesség és magasság.



20. ábra - igazítások

5. Backend

5.1 A Backend konfigurációja

Most, hogy a frontend készenáll, az adatbázis el van készítve következhet a Backend, ami összeköti a Frontendet az adatbázissal. A legelső feladat a backendet önmagát rákapcsolni az adatbázisra, kialakítani és összekapcsolni a táblákat.

Ezután következhetett a kontrollerek kialakítása. Ezt az adatbázis tábláit követve kellett megtenni, ügyelve a kapcsolótábláknál az idegenkulcsok mellett a navigációs egységekre. A Backend feladatai:

- Fogadni a frontend kéréseit.
- Ezen kérés szerint kommunikálni az adatbázissal
- Az eredményt visszaküldeni a frontendnek

5.2 Backendhez használt programok, keretrendszerek

A backendhez a Visual Studio 2022-es programját használtuk, illetve az ASP.NET Core keretrendszert.

A Visual Studio 2022 a C# fejlesztés egyik legerősebb és legkomplexebb professzionális eszköze, amely számos olyan előnyt kínál, ami miatt világszerte az egyik legelterjedtebb választás a fejlesztők körében. Az egyik legnagyobb erőssége, hogy a .NET és a C# ökoszisztémára lett optimalizálva, így szinte minden funkció natívan, magas szinten támogatott benne. A fejlesztők intelligens kódkiegészítést, fejlett szintaxis-ellenőrzést, automatikus refaktorálási lehetőségeket és valós idejű hibajelzéseket kapnak, amelyek jelentősen gyorsítják a munkát és csökkentik a hibák számát. A beépített IntelliSense rendszer nemcsak a kódírást teszi kényelmesebbé, hanem segít a nagyobb projektekben való gyors eligazodásban is.

A Visual Studio 2022 különösen erős a hibakeresés területén. Sok fejlesztő szerint a piac egyik legjobb debuggere található benne, amely részletes breakpoint-kezelést, feltételes

megállításokat, memóriafigyelést, változókövetést, hívási verem elemzést és többszálú alkalmazások vizsgálatát is lehetővé teszi. Ez különösen fontos összetettebb C# alkalmazások fejlesztésekor, ahol a logikai hibák és a futás idejű problémák gyors felderítése kritikus. A diagnosztikai és teljesítményelemző eszközök szintén komoly előnyt jelentenek, hiszen a memóriahasználat, CPU-terhelés vagy lassulások könnyen feltérképezhetők.

A felhasználói felületek fejlesztése szintén kiemelkedően kényelmes benne. A Windows Forms drag-and-drop tervezője gyors asztali alkalmazásfejlesztést tesz lehetővé, míg a WPF és a XAML-alapú fejlesztés vizuális szerkesztőkkel és valós idejű előnézettel támogatott. Emellett a modern, többplatformos fejlesztéshez a .NET MAUI támogatása is rendelkezésre áll, így ugyanazzal a C# tudással akár Windowsra, Androidra, iOS-re vagy macOS-re is készíthetők alkalmazások.

A Visual Studio 2022 előnye továbbá, hogy egyetlen integrált környezetben kínálja a teljes fejlesztési folyamat eszközeit. A projektkezelés, a NuGet csomagkezelés, a verziókezelés és a tesztelés mind közvetlenül beépítve érhető el. A Git-integráció révén egyszerű a branchek kezelése, az összehasonlítás, a merge és a pull request workflow követése. A beépített tesztkezelővel unit tesztek futtathatók és elemezhetők, a code coverage funkcióval pedig mérhető, mennyire lefedett a kódbázis.

Nagyvállalati vagy összetettebb projektek esetén a Visual Studio különösen jól teljesít, mert képes nagy solutionök, sok projektből álló rendszerek és enterprise szintű alkalmazások kezelésére. A Solution Explorer, a fejlett keresőeszközök, a dependency elemzés és az architektúra-támogató funkciók segítenek a nagy rendszerek átlátható kezelésében.

Összességében a Visual Studio 2022 azért számít a C# fejlesztés egyik legjobb választásának, mert nem csupán egy kódszerkesztő, hanem egy komplett professzionális fejlesztői platform. Egyetlen alkalmazásban egyesíti a gyors kódírást, a fejlett hibakeresést, a vizuális felületfejlesztést, a tesztelést, a verziókezelést és a teljesítményoptimalizálást, ami jelentősen növeli a fejlesztés sebességét és minőségét.

Az **ASP.NET Core** keretrendszer egy olyan modern, nagy teljesítményű megoldást ad, ami kifejezetten jól illeszkedik API-alapú backend fejlesztéshez. Már az első pillanattól érezhető, hogy erre a környezetre optimalizálták: gyors, hatékonyan kezeli a párhuzamos kéréseket, és natívan támogatja az aszinkron működést, ami ma már alapelvárás egy webes szolgáltatásnál. Sok más keretrendszerhez képest itt nem kell külön küzdeni a teljesítménnyel, mert alapból jól skálázódik.

Az adatbázis-kezelésnél is kényelmesebb megoldást ad, mivel az **Entity Framework Core** segítségével objektumokon keresztül lehet dolgozni az adatokkal, így nem kell minden lekérdezést kézzel SQL-ben megírni. Ez nemcsak gyorsítja a fejlesztést, hanem csökkenti a hibalehetőségeket is, miközben megmarad a lehetőség arra, hogy szükség esetén alacsonyabb szinten is beavatkozzak.

A választás mellett szólt az is, hogy teljesen platformfüggetlen: ugyanaz az alkalmazás gond nélkül fut Windows, Linux vagy akár konténeres környezetben is. Sok régebbi vagy kötöttebb technológiával szemben itt nincs ilyen megkötés.

Végül pedig az is nagy előny, hogy a keretrendszer rengeteg dolgot készen ad: routing, validáció, JSON-kezelés, biztonsági alapok. Összességében az ASP.NET Core azért jobb választás sok más megoldáshoz képest, mert egyszerre gyors, jól strukturált, könnyen skálázható, és fejlesztőként is hatékonyabb munkát tesz lehetővé.

5.3 Backend és az adatbázis kapcsolata

Mielőtt bármi munkához elkezdnénk, először szükséges az adatbázis elérése, ehhez szükségünk van az adatbázis elérhetőségére, (Például Localhost) majd a portra, amin keresztül csatlakozunk, ami eltérő gépeken eltérő értékű lehet. (Például 3306, 3307 stb…) Ezt követően még meg kell adnunk konkrétan az adatbázis nevét. (Például „pizzaprojekt”.) Ezzel már elértük, hogy a program látja az adatbázist. Vagyis látná, ha be tudna lépni. Ezért a következő adat, amit meg kell adni az a felhasználónév és a jelszó amennyiben felhasználóhoz kötött. Ez után elkezdhetjük felépíteni a struktúrát, a kontrollereket, a táblákat.

```
23    Services.AddControllers(); //Hozz•adom a vez•rl•ket a szolg•ltat•sokhoz a app konfigur•ci•j•ban.
24    Services.AddDbContext<PizzaContext>(opt => opt.UseMySQL("server=localhost;port=3306;database=pizzaprojekt;user=root;")); //Hozz•adom az adatb•zis kontextust a szolg•ltat•sokhoz, •s konfigur•
```

21. ábra - vezérlők

5.4 A táblák felépítése

Már elérjük az adatbázist, vannak adataink. De valahogy ki kell adnunk a frontendnek. Ezért létre kell hozni a kontrollereket, és az ehhez szükséges struktúra hátteret.

Minden kontroller létrehozása előtt fel kell építenünk, hogy a kontroller éppen milyen adatszerkezetből dolgozzon. (Meg kell alkotnunk neki az adatbázis táblájának megfelelőjét.) Megadva azt is, hogy az adott adat az mi pontosan az adatbázisban.

Megadhatjuk, hogy az adat kitöltése kötelező legyen, amit majd a kontrollerben tudunk ellenőrizni.

```
1   using Microsoft.EntityFrameworkCore;
2   using System.ComponentModel.DataAnnotations;
3   using System.ComponentModel.DataAnnotations.Schema;
4
5   namespace pizzaprojekt.Models
6   {
7       [PrimaryKey(nameof(RendeloAzonosito))]
        9 references
8       public class Vasarlo
9       {
10          [Column("Rendelo_azonosito")]
11          [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
            16 references
12          public int RendeloAzonosito { get; set; }
13          [Column("Nev")]
            8 references
14          [Required] public string? Nev { get; set; }
15          [Column("Telefonszam")]
            8 references
16          public string? Telefonszam { get; set; }
17      }
18  }
19
```

22. ábra - Kötelező

5.5 A kontrollerek

```
23      public class VasarloDTO
24      {
25          // Nev kötelező mezőként jelölve
26          [Required]
27          [JsonPropertyName("nev")]
            2 references
28          public string Nev { get; set; } = string.Empty; // alapértelmezett üres stringgel
29          // Telefon opcionális, de ha meg van adva, ellenőrzés: telefonszám formátum
30          [Phone]
31          [JsonPropertyName("telefonszam")]
            2 references
32          public string? Telefonszam { get; set; }=string.Empty;
33      }
34
        5 references
35      public class CimOutDTO
36      {
37          [JsonPropertyName("cimId")]
            7 references
38          public int CimId { get; set; }
39
40          [JsonPropertyName("varos")]
            3 references
41          public string? Varos { get; set; }
42
43          [JsonPropertyName("iranyitoszam")]
            3 references
44          public string? Iranyitoszam { get; set; }
45
46          [JsonPropertyName("utca")]
            3 references
47          public string? Utca { get; set; }
48
49          [JsonPropertyName("hazszam")]
            3 references
50          public string? Hazszam { get; set; }
51
52          [JsonPropertyName("emeletAjto")]
            3 references
53          public string? EmeletAjto { get; set; }
54      }
55
56      // DTO a vásárló lekérdezéséhez / visszaküldéséhez
        6 references
57      public class VasarloOutDTO
58      {
59          [JsonPropertyName("rendeloAzonosito")]
            3 references
60          public int Id { get; set; } // Egyedi azonosító
61          [JsonPropertyName("nev")]
            4 references
62          public string? Nev { get; set; } // Név (nullable)
63          [JsonPropertyName("telefonszam")]
            3 references
64          public string? Telefonszam { get; set; } // Telefon (nullable)
65          [JsonPropertyName("cim")]
            2 references
66          public CimOutDTO? Cim { get; set; } // Legutóbbi cím
67          [JsonPropertyName("cimek")]
            2 references
68          public List<CimOutDTO> Cimek { get; set; } = new(); // Összes mentett cím
```

23. ábra - DTO

Miután felépítettük a táblát a backendben, létrehozhatjuk a kontrollert mellé. Kezdetnek először adatátviteli objektumot azaz DTO-t (Data transferin Object rövidítése) hozunk létre. A DTO lehet magában a kontrollerben is, de általában külön fájlban szinte kizárólagosan elterjedtek. Mivel külön fájlban és külön mappában megőrzi a réteges architektúrát és gyorsan átláthatatlanná válik ahogy a program bővül. Valamint elterjedt, hogy külön veszszük bemeneti és kimeneti DTO-kra. Ezzel nem küldünk vissza véletlen olyan adatot amit nem szabadna, hogy a felhasználó lásson. (Például más jelszavak.) Ezután meg tudjuk írni a végpontokat, mint például az információ lekérés, a változtatás vagy a törlést.

5.6 A frontend és backend kommunikáció

A frontenden elhelyezett UI elemek eltérő kérést küldenek a backendnek. Ezeket a kéréseket fel kell dolgozni, és továbbítani az adatbázis felé a megfelelő táblához, a megfelelő kérést. Majd a kapott választ a frontend felé kell visszaadni. Ezért is fontos a DTO szempontjából,

hogy ne csak egy nagy legyen, hanem egy Request és egy Response DTO is. Ezen kérések a mai sztenderdekben aszinkron kérések, annak érdekében, hogy az app fluid futása biztosított legyen, így eszerint kell eljárni a motorház alatt a backendben is.

```
[HttpPost]
0 references
public async Task<ActionResult<VasarloOutDTO>> Post([FromBody] VasarloDTO vasarloDto)
{
    if (!ModelState.IsValid)
        return BadRequest(ModelState); // Ha a DTO invalid, 400-as hibát küldünk

    // Új Vásárló objektum létrehozása a DTO alapján
    var result = new Vasarlo
    {
        Nev = vasarloDto.Nev, // Név beállítása
        Telefonszam = vasarloDto.Telefonszam, // Telefon beállítása
    };

    _ctx.Vasarlo.Add(result); // Hozzáadjuk az adatbázishoz
    await _ctx.SaveChangesAsync();


    // Visszaadjuk a létrehozott objektumot DTO formában, HTTP 201 státusszal
    return CreatedAtAction(
        nameof(Get), // A GET metódus hivatkozása az új elem lekéréséhez
        new { id = result.RendeloAzonosito }, // Új ID útvonala
        new VasarloOutDTO
        {
            Id = result.RendeloAzonosito,
            Nev = result.Nev,
            Telefonszam = result.Telefonszam
        });
}
```

24. ábra - Kontroller

Mindig igazodni kell a frontendhez, mivel a visszakapott adatokat kívánja megjeleníteni, így egy apró hiba is az app összeomlásához vezethet. Ezért először ellenőrizzük, hogy egyáltalán az elvárt kereteken belüli adatok érkeztek a frontendtől. Amennyiben a megadott kereteken kívül esik, hibakóddal térünk vissza. Ezután ha az adatok megfelelnek, lefuttatjuk a kérést, majd Json formátumban kapott választ továbbítjuk a frontendnek. Ha új pizzát akarunk felvinni, akkor a névnek kötelezően szerepelnie kell, vagy hibát fogunk visszakapni, mert a név adat előtt volt egy [Required] kitétel, ami nem engedi, hogy üresen hagyjuk, ezt ellenőrzi le az If(! Modstate.Isinvalid) kódrész.

6. Működésének műszaki feltételei

Elsődlegesen az egész programot úgy építettük fel, hogy lokálisan helyi hálózaton működjön. Ezáltal nincs szükség internetre a futtatáshoz.

Az elképzeléseink szerint szükség van egy alacsony fogyasztású Windows-os számítógépre, ami a szerver funkcióját tölti ki. Ezen futna minden. A felhasználói felületet meg bármilyen a hálózaton lévő géppel el lehet érni.

A Pulthoz egy egyszerű érintőkijelzős tabletre gondoltunk. Mivel böngészőben lehet elérni a felhasználói felületet, így bármilyen tablet megfelelő. Lehet Androidos vagy Apple-ös rendszer, csak a megfelelő kijelző méretre kell ügyelni (minél nagyobb annál jobb).

A Konyhába is egy egyszerű tabletre gondoltunk.

6.1 Telepítési útmutató

A projekt telepítéséhez az alábbi előfeltételek szükségesek: Node.js (v18 vagy újabb) és npm csomagkezelő, valamint a háttérrendszer futtatásához .NET 7 SDK.

1. **lépés – A forráskód letöltése:**

A projekt könyvtárába lépés után klónozzuk vagy másoljuk a projektet.

2. **lépés – Függőségek telepítése:**

cmd parancs: *npm install*

Ez telepíti az összes szükséges csomagot (React, React Router, Bootstrap, Flatpickr stb.) a package.json alapján.

3. **lépés – Háttérrendszer indítása:**

A React frontend egy .NET alapú REST API-val kommunikál, amely alapértelmezetten a *https://localhost:7195* címen fut. A Vite konfigurációjában proxy van beállítva, amely az */api/** kéréseket automatikusan erre a végpontra irányítja.

4. **lépés – Fejlesztői szerver indítása:**

cmd parancs: *npm run dev*

Az alkalmazás a *http://localhost:5173* címen érhető el.

7. Tesztelések

7.1 Fejlesztés közbeni tesztelések

A program írása közbeni tesztelések elkerülhetetlenek Backend-nél és Frontend-nél is, bár mindkét területen másfajta módszerrel, szemlélettel kell tesztelni. A Backend-nél a fejlesztés közbeni tesztekkel az egyes funkciók működését lehet ellenőrizni. Persze, ha egy funkciót leteszteltünk és működött utána még nem szabad nem tesztelni a letesztelt funkciót később. A program apróbb módosításai hatással lehetnek a már letesztelt funkciókra. Így a fejlesztés közbeni tesztelések inkább csak apróbb visszajelzésként kell elfogadni és a későbbiekben újra tesztelni az egészet.

	teszt jellege	Gubacsi Tamás	Jelencsics Bálint	Kacsó Marcell	Megfelelő?
Swagger tesztek	Feltétek GET	X		X	Igen
	Keri GET	X		X	Igen
	Keri POST	X		X	Igen
	Keri GET id	X		X	Igen
	Kiszállítási hely GET	X		X	Igen
	Kiszállítási hely POST	X		X	Igen
	Kiszállítási hely GET id	X		X	Igen
	Lead POST	X		X	Igen
	Lead GET	X		X	Igen
	Pizza GET	X		X	Igen
	Pizza GET id	X		X	Igen
	Pizza feltétek GET	X		X	Igen
	Pizza feltétek GET id	X		X	Igen
	Rendelés GET	X		X	Igen
	Rendelés POST	X		X	Igen
	Rendelés GET id	X		X	Igen
	Rendelések GET	X		X	Igen
	Rendelések PUT id	X		X	Igen
	Tartalmaz GET	X		X	Igen
	Tartalmaz POST	X		X	Igen
	Tartalmaz GET id	X		X	Igen
	Vasarlo GET	X		X	Igen
	Vasarlo POST	X		X	Igen
	Vasarlo GET id	X		X	Igen

2. táblázat - Backend teszt

Backend-nél általában az adatbázissal való kommunikációt lehetett tesztelni. A fejlesztés közbeni tesztelést nincs értelme dokumentálni, csak annyit érdemes, hogy milyen funkció tesztet végeztünk, de az eredmény nélkül. A kész programot utána ezek a pontok alapján is át kell nézni és tesztelni újra. A Backend program POST és GET metódusokkal van teli. Minden táblához kell egy-egy metódus. A mi adatbázisunk 10 táblából áll, így minimum 20 metódust kellett írni, és ezeket egyenként letesztelni. Több táblánál is szükség van az id alapján működő GET metódusra. A programunkban nincs szükség törlés és módosítás funkcióra, így azok nem lettek bele építve.

A Frontend-nél is fejlesztés közbeni teszteléseket hajtottunk végre, de itt is az eredmények dokumentálása helyett csak a tesszteket írtuk össze, hogy majd a kész programon egyben teszteljünk mindent.

7.2 Statikus tesztelések

A végső teszteléseknél elővettük az összeírt táblázatunkat. A teszteléseket feladatonként 2 embernek külön-külön kellett lefuttatnia. A tesztek után összegeztük az eredményeket és megbeszéltük, hogy hol találtunk hibákat.

	teszt jellege	Gubacsi Tamás	Jelencsics Bálint	Kacsó Marcell	Megfelelő?
Frontend tesztek	Pizza kiválasztás	X	X		Igen
	Pizza kiválasztás - mégse	X	X		Igen
	Pizza összes feltét hozzáadása	X	X		Igen
	Csak pizza kiválasztása	X	X		Igen
	Rendelésfelvétel: név üresen hagyása	X	X		Igen
	Rendelésfelvétel: telefonszám üresen hagyása	X	X		Igen
	Rendelésfelvétel: cím részleges üresen hagyása	X	X		Igen
	Rendelésfelvétel: Pizza mennyiség változtatása	X	X		Igen
	Rendelésfelvétel: pizza törlése a listából	X	X		Igen
	Rendelésfelvétel: egész rendelés törlése	X	X		Igen
	Konyha: szűrők tesztelése	X	X		Igen
	Konyha: sütés elindítása	X	X		Igen
	Konyha: sütés számláló folytonossága	X	X		Igen
	Konyha: sütés kész gomb tesztelése	X	X		Igen
	Konyha: állapotok változtatása	X	X		Igen
	Konyha: állapotok változtatásánál a sorrend megfelelőséges	X	X		Igen
	Átadás: Rendlési információk megjelennek	X	X		Igen
	Átadás: nyomtatás gomb működése	X	X		Igen
	Átadás: átadás gomb működése	X	X		Igen
	Átadás: dátum változtatás, adatok lekérése	X	X		Igen
	Átadás: korábbi rendelésnél adatok nyomtatása	X	X		Igen
	Átadás: átadás gomb megnyomása után a rendelés áthelyezése az átadott listába	X	X		Igen

3. táblázat - Frontend teszt

7. Továbbfejlesztési lehetőségek

Több fejlesztési lehetőséget is találtunk még a munka folyamán:

1. Ha az egész programot távolról is elérhetővé tesszük, akkor akár a futárok is láthatják a rendeléseket és hogy hova kell kiszállítani. Ezáltal még jobban csökkenthetőek lennének a felesleges papírok száma.
2. POS terminál készítése: a vásárló adhatja le a rendelést egy érintőkijelző segítségével. (Lásd: McDonald's)
3. Mobil rendelés létrehozása: bárki vehet fel rendelést interneten keresztűl, akár úgy, hogy házhoz kéri a rendelést vagy helyben fogyasztással.
4. Statisztikák készítése: az adatbázisnak hála egyszerű statisztikákat lehet csinálni. Például: leggyakrabban fogyó pizzák, legtöbbet használt feltét. Ezek nem csak felesleges adatok, tovább elemezve az egészet költségmegtakarítást lehet elérni. Például, ha kielemezzük, hogy szezonálisan mely pizzák és feltétek fogynak, így láthatjuk előre, hogy miből mennyi kell a hónapok folyamán.

Természetesen a fejlesztési lehetőségeknek csak a képzeleterő hiánya szabhat határt.

Ábrajegyzék

1. táblázat - feladatok 4
2. táblázat - Backend teszt 24
3. táblázat - Frontend teszt 25

1. ábra - OneDrive felhő 4
2. ábra – ER Modell 6
3. ábra - adatbázis 9
4. ábra - kezdeti terv 9
5. ábra - Json adat 10
6. ábra - Főoldal 11
7. ábra - Feltét választó 11
8. ábra - rendelés rögzítés 12
9. ábra - Konyha oldal 12
10. ábra - státusz szűrők 12
11. ábra - Visszaszámláló 13
12. ábra - feladatok rendezése 14
13. ábra - Rendelések 15
14. ábra - Kiszállítási adatok 15
15. ábra - korábbi rendelések 16
16. ábra - Nyomtatási kép 16
17. ábra - címek 16
18. ábra - plusz funkciók 17
19. ábra - szinezések 17
20. ábra - igazítások 18
21. ábra - vezérlők 20
22. ábra - Kötelező 21
23. ábra - DTO 21
24. ábra - Kontroller 22

Tartalom

1. Bevezetés 1

1.1 A program célja 1

1.2 Felhasználási terület - célközönség 1

1.3 Alap folyamat – kiinduló állapot 1

1.4 Hibázási lehetőségek 2

1.5 Optimalizálás – a program funkciói 2

2. A program 3

2.1 Felhasználói igények 3

2.2 Tervezés: projekt tagok 3

2.3 Tervezés: feladatok felosztása: 3

2.4 Kapcsolattartás módszere 4

3. Adatbázis – ER modell megtervezése 4

3.1 ER modell: 6

3.2 Az adatbázis cellái 6

3.3 Adatbázis létrehozása SQL-ben 7

3.4 Adatbázis feltöltése 8

3.5 A kész adatbázis 9

4. Frontend 9

4.1 Frontend tervezése 9

4.2 Frontendhez használt programok, keretrendszerek 10

4.3 Kezdőlap – Rendelés rögzítés 11

4.4 Konyha – készültségi állapotok 12

4.5 Rendelések átadás - kiszállítás 15

4.6 Végső igazítások 16

5. Backend 18

5.1 A Backend konfigurációja 18

5.2 Backendhez használt programok, keretrendszerek 18
5.3 Backend és az adatbázis kapcsolata 20
5.4 A táblák felépítése 20
5.5 A kontrollerek 21
5.6 A frontend és backend kommunikáció 21
6. Működésének műszaki feltételei 22
7. Tesztelések 23
7.1 Fejlesztés közbeni tesztelések 23
7.2 Statikus tesztelések 25
7. Továbbfejlesztési lehetőségek 26
Ábrajegyzék 27